

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Pseudocode Writing Task (Algorithm Design)

Name: Prabhavathi R Register No: 192321101

Course: Computer Vision Course Code: ITA05 CO5

Assessment Tool Weightage: 20%

CO5: Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. The answers below follow the 15 pseudocode tasks in the supplied Assessment Tool 2 and use modular stages, control flow, and efficiency considerations as requested.

1. Real-Time Face Detection and Recognition Pipeline: Write detailed pseudocode for continuous live capture, face detection, recognition, and display while handling multiple faces efficiently.

```
MODULE LoadModels() faceDetector <- LoadOpenCVFaceDetector() recognizer <-  
LoadPretrainedFaceRecognizer() RETURN faceDetector, recognizer END MODULE  
MODULE Preprocess(frame)  
smallFrame <- Resize(frame) grayFrame <- ConvertToGray(smallFrame) grayFrame <-  
ApplyLightContrastNormalization(grayFrame) RETURN grayFrame END MODULE  
DetectAndRecognize(grayFrame, frame, detector, recognizer) faces <- detector.Detect(grayFrame) FOR each face IN  
faces roi <- Crop(grayFrame, face) roi <- Resize(roi,  
recognizerInputSize) (label, confidence) <- recognizer.Predict(roi) IF confidence passes threshold THEN text <- label  
ELSE text <- "Unknown" END IF DrawRectangle(frame, face) DrawText(frame, text) END FOR  
END MODULE  
MAIN  
detector, recognizer <- LoadModels() camera <- OpenCamera() WHILE camera is open  
frame <- ReadFrame(camera)  
IF frame is invalid THEN BREAK  
gray <- Preprocess(frame) DetectAndRecognize(gray, frame, detector, recognizer)  
Display(frame) IF ExitKeyPressed() THEN BREAK  
END WHILE  
ReleaseCamera() END MAIN
```

2. OCR Pipeline for Document Processing: Write pseudocode for batch reading, enhancement, segmentation, and text extraction from noisy, low-contrast documents.

```
MODULE Preprocess(image) gray <- ConvertToGray(image) gray <- Denoise(gray) enhanced <- ApplyCLAHE(gray)  
binary <- AdaptiveThreshold(enhanced) binary <- MorphologicalOpenClose(binary) RETURN binary END MODULE  
MODULE ProcessDocument(path) image <- ReadImage(path) IF image is invalid THEN RETURN cleanImage  
<Preprocess(image) regions <- SegmentTextRegions(cleanImage) fullText <- "" FOR each region IN regions  
text <- OCR(region) fullText <- fullText + text END FOR  
SaveText(path, fullText) DisplayText(fullText) END MODULE  
MAIN  
files <- GetImageFiles(inputFolder) FOR each file IN files  
ProcessDocument(file) ReleaseTemporaryImages() END FOR  
END MAIN
```

3. Motion Detection and Tracking System: Write pseudocode including framedifferencing/filtering, tracking, visualization, dynamic backgrounds, and continuous video control.

```
MAIN  
video <- OpenVideo() backgroundModel <- InitializeAdaptiveBackgroundModel() tracker <- InitializeTracker()  
WHILE video is open  
frame <- ReadFrame(video) IF frame is invalid THEN BREAK  
gray <- ConvertToGray(frame)  
gray <- GaussianBlur(gray) foreground <- BackgroundSubtract(backgroundModel, gray) mask <-  
Threshold(foreground) mask <- MorphologicalOpenClose(mask) regions <- FindContours(mask) detections <- [] FOR
```

```

each region IN regions IF Area(region) >= MinimumArea THEN detections.ADD(BoundingBox(region)) END IF END
FOR tracks <-
tracker.Update(detections) FOR each track IN tracks DrawRectangle(frame, track.box) DrawText(frame, track.id)
END
FOR Display(frame) IF ExitKeyPressed() THEN BREAK END WHILE ReleaseVideo() END MAIN

```

4. Vehicle Counting System using Video: Write pseudocode for ROI detection, tracking, counting, double-count prevention, and visualization.

```

MAIN video <- OpenVideo() detector <- LoadVehicleDetector() tracker <- InitializeTracker() countedIDs <- EmptySet()
count <- 0 countingLine <- DefineLine() WHILE video is open frame <- ReadFrame(video) IF frame is invalid THEN
BREAK roi <- CropToROI(frame) detections <- detector.Detect(roi) tracks <- tracker.Update(detections) FOR each
track IN tracks DrawRectangle(frame, track.box) DrawText(frame, track.id) IF CrossedLine(track.previousCenter,
track.currentCenter, countingLine) AND track.id NOT IN countedIDs THEN count <- count + 1
countedIDs.ADD(track.id)
END IF END FOR DrawLine(frame, countingLine) DrawText(frame, "Vehicle Count: " + count) Display(frame) IF
ExitKeyPressed() THEN BREAK END WHILE ReleaseVideo() END MAIN

```

5. Facial Expression Recognition System: Write efficient pseudocode for camera capture, face detection, feature extraction, classification, and display.

```

MODULE ClassifyExpression(faceROI) normalized <- NormalizeFace(faceROI) features <-
ExtractCompactFacialFeatures(normalized) expression <- ExpressionClassifier.Predict(features) RETURN
expression
END MODULE MAIN camera <- OpenCamera() faceDetector <- LoadLightweightFaceDetector() classifier <-
LoadExpressionClassifier() previousResult <- "neutral" WHILE camera is open frame <- ReadFrame(camera) IF
frame is invalid THEN BREAK small <- Resize(frame) gray <- ConvertToGray(small) faces <-
faceDetector.Detect(gray) FOR each face IN faces roi <- Crop(gray, face) result <- ClassifyExpression(roi) result <-
TemporalSmooth(result, previousResult) previousResult <- result DrawRectangle(frame, face) DrawText(frame,
result) END FOR Display(frame) IF ExitKeyPressed() THEN BREAK END WHILE ReleaseCamera() END MAIN

```

6. Real-Time Object Detection Pipeline: Write detailed pseudocode for efficient multi-object detection, visualization, and looping.

```

MAIN camera <- OpenCamera() model <- LoadTrainedOpenCVDNNModel() WHILE camera is open frame
<- ReadFrame(camera) IF frame is invalid THEN BREAK input <- ResizeAndNormalize(frame) detections
<- model.Detect(input) detections <- FilterByConfidence(detections) detections <-
NonMaximumSuppression(detections)
FOR each detection IN detections DrawRectangle(frame, detection.box) DrawText(frame, detection.label) END FOR
Display(frame) IF ExitKeyPressed() THEN BREAK END WHILE ReleaseCamera() END MAIN

```

7. Image Difference Detection System: Write pseudocode to read two images, preprocess them, compare them accurately, handle noise, and visualize differences.

```

MAIN image1 <- ReadImage(file1) image2 <- ReadImage(file2) IF either image is invalid THEN STOP image1
<- ResizeToSameSize(image1, image2) gray1 <- ConvertToGray(image1) gray2 <- ConvertToGray(image2) gray1
<- Denoise(gray1) gray2 <- Denoise(gray2) difference <- AbsoluteDifference(gray1, gray2) mask <-
Threshold(difference) mask <- MorphologicalOpenClose(mask) contours <- FindContours(mask) result <-
Copy(image2) FOR each contour IN contours IF Area(contour) >= MinimumArea THEN box <- BoundingBox(contour)
DrawRectangle(result, box) END IF END FOR Display(result) END MAIN

```

8. Video Frame Extraction System: Write structured pseudocode to read video, extract frames at fixed intervals, save selected frames efficiently, and avoid redundancy.

```
MAIN video <- OpenVideo(inputVideo) interval <- DesiredFrameInterval() frameNumber <- 0 saveNumber <- 0
WHILE video is open frame <- ReadFrame(video) IF frame is invalid THEN BREAK IF frameNumber MOD interval =
0 THEN IF
IsSufficientlyDifferent(frame, previousSavedFrame) THEN filename <- CreateFrameFilename(saveNumber)
SaveImage(filename, frame) previousSavedFrame <- frame saveNumber <- saveNumber + 1 END IF END IF
frameNumber <- frameNumber + 1 END WHILE ReleaseVideo() END MAIN
```

9. Background Subtraction Pipeline: Write pseudocode for background modeling, subtraction, filtering, highlighting, and display while handling dynamic backgrounds.

```
MAIN video <- OpenVideo() background <- InitializeMOG2orKNN() WHILE video is open frame <- ReadFrame(video)
IF frame is invalid THEN BREAK foreground <- background.Apply(frame) mask <- Threshold(foreground) mask
<MorphologicalOpenClose(mask) contours <- FindContours(mask) FOR each contour IN contours IF Area(contour)
>=
MinimumArea THEN box <- BoundingBox(contour) DrawRectangle(frame, box) END IF END FOR Display(frame) IF
ExitKeyPressed() THEN BREAK END WHILE ReleaseVideo() END MAIN
```

10. Edge Detection Pipeline for Real-Time Video: Write pseudocode with preprocessing, edge detection, visualization, and real-time control.

```
MAIN camera <- OpenCamera() WHILE camera is open frame <- ReadFrame(camera) IF frame is invalid THEN
BREAK small <- Resize(frame) gray <- ConvertToGray(small) blurred <- GaussianBlur(gray) edges <- Canny(blurred,
lowThreshold, highThreshold) Display(edges) IF ExitKeyPressed() THEN BREAK END WHILE ReleaseCamera()
END MAIN
```

11. Multi-Image Batch Processing System: Write pseudocode to read images from a folder, preprocess and detect features, store results, and manage memory efficiently.

```
MODULE ProcessImage(path) image <- ReadImage(path) IF image is invalid THEN RETURN gray <-
ConvertToGray(image) clean <- Denoise(gray) features <- DetectFeatures(clean) result <- CreateResultRecord(path,
features) SaveResult(result) Release(image, gray, clean, features) END MODULE MAIN files
<GetImageFiles(inputFolder) FOR each file IN files ProcessImage(file) END FOR END MAIN
```

12. Face Detection with ROI Extraction: Write pseudocode to detect multiple faces, crop their ROIs, save them, and avoid redundant processing.

```
MAIN image <- ReadImage(inputImage) gray <- ConvertToGray(image) detector <- LoadFaceDetector() faces
<detector.Detect(gray) index <- 0 seenRegions <- EmptyList() FOR each face IN faces IF NotRedundant(face,
seenRegions) THEN roi <- Crop(image, face) roi <- Resize(roi) SaveImage("face_" + index + ".jpg", roi)
seenRegions.ADD(face) index <- index + 1 END IF END FOR DisplayDetectedFaces(image) END MAIN
```

13. Video Stabilization Pipeline: Write pseudocode for reading frames, motion estimation, transformation, stabilization, and display while reducing jitter in real time.

```
MAIN video <- OpenVideo() previousGray <- NULL previousTransform <- IdentityTransform() WHILE video is open
frame <- ReadFrame(video) IF frame is invalid THEN BREAK gray <- ConvertToGray(frame) IF previousGray is
NULL
THEN stabilized <- frame ELSE pointsPrev <- DetectMotionFeatures(previousGray) pointsCurr <-
```

```

TrackFeatures(previousGray, gray, pointsPrev) transform <- EstimateMotionTransform(pointsPrev, pointsCurr)
transform <- SmoothTransform(transform, previousTransform) stabilized <- WarpFrame(frame, transform)
previousTransform <- transform END IF previousGray <- gray Display(stabilized) IF ExitKeyPressed() THEN BREAK
END WHILE ReleaseVideo() END MAIN

```

14. Text Detection in Natural Scene Images: Write pseudocode for preprocessing, text-region detection, highlighting, and handling varying backgrounds.

```

MAIN files <- GetInputImages() FOR each file IN files image <- ReadImage(file) gray <- ConvertToGray(image)
enhanced <- ImproveContrast(gray) filtered <- Denoise(enhanced) textMask <- DetectTextRegions(filtered) textMask
<-
MorphologicalFiltering(textMask) regions <- FindTextContours(textMask) FOR each region IN regions IF
RegionPassesTextConstraints(region) THEN DrawRectangle(image, region) END IF END FOR Display(image) END
FOR END MAIN

```

15. Feature Matching Pipeline: Write pseudocode to read two images, extract scale/rotation-robust features, match them efficiently, and display matches.

```

MAIN image1 <- ReadImage(file1) image2 <- ReadImage(file2) gray1 <- ConvertToGray(image1) gray2
<- ConvertToGray(image2) detector <- InitializeORB() keypoints1, descriptors1 <- detector.DetectAndCompute(gray1)
keypoints2, descriptors2 <- detector.DetectAndCompute(gray2) matcher <-
InitializeBFMatcherWithHammingDistance() matches <- matcher.Match(descriptors1, descriptors2) matches <-
SortByDistance(matches) goodMatches <- SelectBestMatches(matches) output <- DrawMatches(image1, keypoints1,
image2, keypoints2, goodMatches) Display(output) END MAIN ORB is suitable because its binary descriptors and
keypoint design provide efficient matching with useful robustness to scale and rotation for many practical scenes.

```

Student Declaration

I, Prabhavathi R (192321101) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

Signature of the student: _____Prabhavathi R__